

Design

This program implements a simple MIP network layer on top of Ethernet raw sockets. Each MIP daemon communicates with local applications through a UNIX SOCK_SEQPACKET socket.

This MIP header is 4 bytes long:

- Destination MIP address: 8 bits
- Source MIP address: 8 bits
- TTL: 4 bits
- SDU length: 9 bits, measured in 32-bit words
- SDU type: 3 bits

The SDUs are padded to a multiple of four bytes before sending and MIP packets are carried in Ethernet frames using EtherType 0x88B5.

MIP-ARP

The daemon keeps an ARP cache containing MIP-address, Ethernet MAC address, and outgoing interface index.

When a destination MAC address is unknown, the daemon sends a MIP-ARP request as an Ethernet broadcast on all active Ethernet interfaces. The MIP destination address is broadcast on address 255. A host receiving a request for its own MIP address sends a unicast MIP-ARP response on the interface where the request was received.

Ping Protocol

The ping client sends PING: to the daemon. The ping server receives the payload and returns PONG:. The client waits up to one second for the matching response and then prints timeout if no response arrives.

Testing

The supplied A-B-C topology was tested:

- A to B: successful ping response
- B to C: successful ping response
- C to B: successful ping response
- A to C: timeout
- C to A: timeout

The A to C and the C to A are expected to print timeout as the implementation does not forward MIP through B.

AI declaration

AI models were used as a learning aid to explain the MIP header bit packing and to help identify implementation issues. The code was reviewed, built and tested myself with the supplied Mininet topology.